Bitácora técnica del reto SDET: Shady Meadows B&B

Índice

1. **Contexto, objetivos y entregables** **2**
2. **Paso 1: Preparación del entorno (instalación y verificación)** **2**
3. **Paso 2: Organización del trabajo (TODO.md)** **3**
4. **Paso 3: Exploración manual de la API con Postman** **4**
5. **Paso 4: Automatización API con Karate (Maven)** **4**
6. **Paso 5: Automatización UI con Playwright (Node)** **5**
7. **Tabla de incidencias (síntoma, causa raíz, solución y verificación)** **5**
8. **Registro de commits relevantes** **5**
9. **Evidencias recomendadas (para anexos)** **5**

1. Contexto, objetivos y entregables

Contexto. El reto solicita validar dos capas del sistema:

- **Capa API:** automatización de endpoints con **Karate DSL** (framework sobre Java/JVM).
- **Capa UI:** automatización de flujos críticos de la interfaz web con **Playwright** (Node.js).

Objetivo general. Construir un repositorio reproducible y profesional, con pruebas atómicas, reportes y documentación suficiente para que cualquier evaluador pueda clonarlo y ejecutarlo.

Entregables esperados.

- Suite API (Karate) ejecutable con Maven y con reporte HTML.
- Suite UI (Playwright) ejecutable con npm/npx y con reporte HTML.
- Evidencia de ejecución (reportes) y un README con instrucciones claras.
- Repositorio limpio: sin artefactos generados versionados (p. ej. `target/`, `node_modules/`, reportes).

2. Paso 1: Preparación del entorno (instalación y verificación)

Propósito. Asegurar herramientas mínimas para: control de versiones, ejecución de Karate (Java/Maven) y ejecución de Playwright (Node/npm).

Herramientas instaladas y para qué se usan

- **Git:** control de versiones; permite commits trazables, revertir cambios y entregar un repo profesional.
- **Java (JDK):** runtime requerido por Karate (Karate ejecuta sobre la JVM).
- **Maven:** construcción/ejecución del módulo Java; corre los tests Karate con `mvn test`.
- **Node.js:** runtime requerido por Playwright (tests UI se ejecutan con Node).
- **npm:** gestor de paquetes Node; instala dependencias y ejecuta `npx playwright`.
- **VS Code:** editor principal para crear/modificar features Karate y tests Playwright.

Verificación del entorno

Se verificó que cada herramienta se pudiera invocar desde PowerShell (esto prueba que `PATH` e instalación están correctos):

```
git --version
java -version
mvn -v
node -v
npm -v
```

Incidencias y soluciones (entorno)

- **Git/Maven/Node no reconocidos inicialmente.**
  - **Causa:** herramientas no instaladas o no agregadas al `PATH`.
  - **Solución:** instalación y corrección de variables de entorno.
  - **Verificación:** comandos `-version` responden correctamente.
- **Maven no encontraba `mvn.cmd`.**
  - **Causa:** `MAVEN_HOME` apuntaba a una ruta incorrecta.
  - **Solución:** ajustar `MAVEN_HOME` e incluir `%MAVEN_HOME%\bin` en `PATH`.
  - **Verificación:** `mvn -v` muestra versión y Maven home.
- **npm falló por Execution Policy en PowerShell.**
  - **Causa:** la política de ejecución bloqueaba `npm.ps1`.
  - **Solución:** habilitar política adecuada para el usuario (p. ej. `RemoteSigned`) o usar un shell compatible.
  - **Verificación:** `npm -v` funciona sin error.

3. Paso 2: Organización del trabajo (TODO.md)

Propósito. Formalizar el plan del reto como checklist: definición de *done*, tareas API/UI, reportes y entrega.

Se creó `TODO.md` en la raíz del repositorio para:

- dar transparencia del progreso,
- asegurar consistencia (no olvidar piezas),
- demostrar enfoque de ingeniería (planificación + ejecución).

4. Paso 3: Exploración manual de la API con Postman

Justificación. El PDF del reto no exige Postman, pero explorar manualmente antes reduce riesgo: confirma rutas reales, estructura JSON, campos obligatorios y códigos de respuesta esperados.

Configuración en Postman

Se creó un *Environment* llamado `ShadyMeadows` con variables:

- `baseUrl` = https://automationintesting.online
- `apiBaseUrl` = https://automationintesting.online/api

Incidencia clave: prefijo `/api`

- **Síntoma:** `GET {{baseUrl}}/branding/` devolvió `404 Not Found`.
- **Diagnóstico:** los endpoints válidos operan bajo `/api`.
- **Solución:** usar `{{apiBaseUrl}}/branding/`, `/room/`, `/booking/`.

Requests ejecutadas y hallazgos

GET Branding: `GET {{apiBaseUrl}}/branding/` → `200 OK`. **GET Rooms:** `GET {{apiBaseUrl}}/room/` → `200 OK`. Campos relevantes observados (ejemplo):

```
{
  "roomid": 1,
  "roomPrice": 100
}
```

POST Booking: `POST {{apiBaseUrl}}/booking/` con JSON → `201 Created`. Respuesta devuelve `bookingid` y eco del payload.

5. Paso 4: Automatización API con Karate (Maven)

Propósito y criterio de pruebas atómicas

Se implementó una suite que:

- cubre `GET /branding/`, `GET /room/`, `POST /booking/`,
- genera evidencia mediante reporte HTML,
- es **atómica**: obtiene `roomid` en el mismo flujo y evita depender de estado previo.

Ejecución estándar y reporte

```
cd api-tests/api-tests
mvn clean test
```

Reporte HTML:

- file:///C:/Dev/sdet-challenge/api-tests/api-tests/target/karate-reports/karate-summary.html

6. Paso 5: Automatización UI con Playwright (Node)

Propósito

Validar flujos críticos desde la perspectiva del usuario:

- **Home sanity:** Contact visible + botones `Book this room`.
- **Admin login:** acceso a `/admin`, login `admin/password` y `Logout` visible.
- **Reporte HTML:** evidencia reproducible.

Ejecución estándar y reporte

```
cd ui-tests
npx playwright test
npx playwright show-report
```

7. Tabla de incidencias (síntoma, causa raíz, solución y verificación)

7.1. Registro de commits relevantes

Cómo obtenerlos: desde la raíz del repositorio ejecutar:

```
git log --oneline -10
```

Nota: Reemplaza los *placeholders* de hash por los hashes reales de tu repositorio (los que te muestra `git log -oneline`).

7.2. Evidencias recomendadas (para anexos)

Para fortalecer la entrega, se recomienda anexar:

- Capturas de Postman: `GET branding`, `GET rooms`, `POST booking` (status + body).
- Captura del reporte Karate (`karate-summary.html`) con 3 features en verde.

- Captura del reporte Playwright (`show-report`) con los 2 tests en estado *passed*.
- Captura de `git status` (repo limpio) y `git log -oneline` (historial).

8. Registro de commits relevantes

Cómo obtenerlos:

```
git log --oneline -10
```

Instrucción: completa los hashes reales en las líneas de abajo.

9. Evidencias recomendadas (para anexos)

- Capturas de Postman: `GET branding`, `GET rooms`, `POST booking`.
- Captura del reporte Karate con 3 features en verde.
- Captura del reporte Playwright con tests en *passed*.
- Captura de `git status` (repo limpio) y `git log -oneline`.

ID	Síntoma observado	Causa raíz (diagnóstico)	Solución aplicada + verificación
INC-01	`git`, `mvn` o `node` "no se reconoce"	Herramientas no instaladas o no presentes en `PATH`.	Instalación y verificación con `git -version`, `mvn -v`, `node -v`.
INC-02	`mvn` no se reconoce / Maven no corre	`MAVEN_HOME` apuntaba a carpeta incorrecta (no existía `\bin\mvn.cmd`).	Corrección de `MAVEN_HOME` + `PATH`. Verificado con `mvn -v`.
INC-03	`npm` falla por ExecutionPolicy (PowerShell)	PowerShell bloquea scripts (`npm.ps1`) por política de ejecución.	Ajuste de política (`RemoteSigned`) y/o uso de `npm.cmd`. Verificado con `npm -v`.
INC-04	Postman: `GET /branding/` devuelve 404	Endpoints reales operan bajo prefijo `/api` (ruta incorrecta inicialmente).	Se definió `apiBaseUrl = .../api`. Verificado: `GET /api/branding` → 200.
INC-05	Karate: `baseUrl is not defined`	`karate-config.js` no estaba en el classpath del módulo ejecutado (estructura duplicada `api-tests/api-tests`).	Reubicación/copia de `karate-config.js` al módulo correcto. Verificado al ejecutar `mvn test`.
INC-06	Reporte Karate no se encontraba en la ruta esperada	Ejecución sin limpiar o búsqueda en módulo equivocado; reportes se generan bajo `target/`.	Estandarización con `mvn clean test`. Verificación: existe `target/karate-reports/karate-summary.html`
INC-07	`POST /booking` intermitente: status 409 en vez de 201	Sandbox público con estado variable / conflictos de booking.	Mitigación: fechas dinámicas futuras (reduce choques). Verificado: suite estable (BUILD SUCCESS).
INC-08	Git mostraba cambios en `target/test-classes`	Artefactos generados por Maven fueron rastreados o aparecían como untracked.	Se añadió `target/` a `.gitignore` y se retiró del tracking con `git rm -cached`. Verificado: `git status` limpio.
INC-09	Playwright: estructura duplicada `ui-tests/ui-tests/tests`	Creación accidental de carpeta duplicada al generar/mover archivos.	Consolidación recomendada en `ui-tests/tests`. Verificado: `npx playwright test` ejecuta desde

Commit (hash corto)	Descripción / propósito
_____	**chore: initialize project with TODO checklist** Creación de estructura inicial, checklist y primer punto de control del repo.
_____	**test(api): add Karate API tests for branding, rooms, booking** Implementación inicial de features y runner para endpoints requeridos.
_____	**chore: ignore build artifacts (target/) and clean repo** Agrega reglas de `.gitignore` y elimina `target/` del tracking para mantener repo limpio.
_____	**chore(api): fix Karate config location for baseUrl** Corrección del classpath al ubicar `karate-config.js` en el módulo ejecutado por Maven.
_____	**test(api): stabilize booking test and ignore build artifacts** Ajustes finales al booking para estabilidad + verificación de ignores.
_____	**test(ui): add Playwright UI tests for home and admin** Implementación de pruebas UI (Home sanity y Admin login) + reporte Playwright.

Cuadro 2: Commits principales del proyecto (completa los hashes con los valores reales del repositorio).

Commit (hash corto)	Descripción / propósito
_____	**chore: initialize project with TODO checklist**.
_____	**test(api): add Karate API tests for branding, rooms, booking**.
_____	**chore: ignore build artifacts (target/) and clean repo**.
_____	**test(ui): add Playwright UI tests for home and admin**.

Cuadro 3: Commits principales del proyecto (completa los hashes con los valores reales).